COSC 461 - Project 1: Eagle Delivery

Faizah Fatima | October 26, 2026

1. Project introduction

For this project, I worked with a simulated delivery robot on a campus map. The robot starts at one building and needs to reach another building. I used BFS and A* to plan routes. I also used Q-learning so the robot could learn from rewards and actions. Finally, I tested a custom campus map and compared different exploration rates.

2. Part A: BFS and A* search

The state is the robot's position, shown as a (row, column) pair. The starting state is the selected building, and the goal is the delivery building. The robot can move up, down, left, or right. A wall blocks movement, and every valid move has the same cost of one. The goal test checks whether the robot has reached the destination.

BFS checks locations in order of distance from the start. It finds a shortest route because every move costs the same. A* uses the current route cost plus Manhattan distance to choose which location to check next. Manhattan distance is the number of row moves plus column moves between two positions. It is admissible because it ignores walls, so it cannot overestimate the shortest real route.

BFS and A* results

Map	Task	BFS length	BFS nodes	A* length	A* nodes
Baseline	S to W	18	67	18	22
Baseline	L to D	22	79	22	62
Baseline	H to P	16	41	16	16
Detour	S to W	26	70	26	51
Detour	L to D	22	65	22	60
Detour	H to P	16	41	16	16

Both methods found routes of equal length for all six tasks. A* expanded fewer nodes each time. The S-to-W route grew from 18 steps to 26 steps when the walkway closure was added, so the detour made that delivery harder.

3. Part B: Learning a delivery policy with Q-learning

In Q-learning, the robot tries actions and updates a Q-value for each state and action. A Q-value estimates how useful an action is. I used an epsilon-greedy policy: with probability epsilon, the robot chooses a random action to explore. Otherwise, it chooses an action with the best current Q-value. An unseen action starts with value zero.

The robot gets -1 for each action and an extra 40 when it first reaches its goal. A shorter route therefore gives a better total reward. The Q-learning update moves the old Q-value toward the observed reward plus the discounted best future value. At a terminal state, there is no future reward term.

Training and evaluation settings

Setting	Value
Training episodes	2,000 per seed
Random seeds	0, 1, 2
Evaluation episodes	100 per seed
Maximum actions	200 per episode
Learning rate (alpha)	0.3
Discount factor (gamma)	0.95
Exploration rate (epsilon)	0.2
Movement slip probability	0.0

Final Q-learning evaluation

Task	Success rate	Mean return	Successful steps
Baseline S to W	100%	22	18
Baseline L to D	100%	18	22
Baseline H to P	100%	24	16
Detour S to W	100%	14	26

Each of the three independent seeds reached a 100% success rate on these tasks in the final evaluation. The successful step counts matched the shortest route lengths found with search. The detour S-to-W task took 26 steps and gave a lower mean return of 14. These results show that the learned policy worked well under the tested settings, but they do not prove it would work as well on every different map.

4. Part C: Campus map and exploration experiment

For the custom-map experiment, I tested a campus layout with six building entrances, open walkways, blocked cells, turns, and dead ends. The route from Student Center to Walton Hall went around obstacles rather than straight through them. On this map, BFS found a 20-step route and expanded 58 nodes. A* also found a 20-step route but expanded only 34 nodes.

My question was: how does the exploration rate change learning and final performance? I predicted that epsilon = 0.2 might give a good balance between exploring different paths and following the best-known actions. I compared epsilon values of 0.05, 0.2, and 0.5. I kept the same map, alpha, gamma, episode counts, and seeds 0, 1, and 2.

Final evaluation for the three epsilon settings

Epsilon	Success rate	Mean return	Successful steps
0.05	100%	20	20
0.20	100%	20	20
0.50	100%	20	20

Early learning results

Epsilon	Training success, episodes 1-100	Training success, episodes 101-200
0.05	91.33%	100%
0.20	90.00%	100%
0.50	89.00%	100%

In the first 100 training episodes, epsilon = 0.05 had the highest success rate, followed by 0.2 and 0.5. These differences were small. All three settings reached 100% success in the next 100-episode block. By the final evaluation, all settings succeeded and used 20 steps on average. This means that different exploration rates did not change final performance in this experiment. It does not mean exploration never matters, because the early training returns were different.

5. What I learned

I learned that A* can find a shortest route while checking fewer locations than BFS. I also learned that Q-learning can improve through trial and error until it finds a good delivery policy. My epsilon experiment did not show a difference in final success, but it helped me understand why it is important to look at training progress as well as final results. A limitation is that I used only one custom map, three seeds, and no movement slip. Different maps or uncertain movement might give different results.

The Python code, map files, raw training and evaluation CSVs, summaries, and learning-curve CSVs are included in the project repository.